

1. ок ето виж кво намерих аз и не казвам че е най оптималния и добър начин искам просто мнението ти
2. **ChatGPT каза:**

Разбира се — точно така е най-добре. 👍

Прати ми каквото си намерил. Няма да приемам автоматично, че е правилното решение.

Ще го разгледаме спокойно и ще ти кажа:

- **как работи;**
- **какъв проблем решава;**
- **какви са плюсовете и минусите;**
- дали е подходящо за **нашия конкретен проект/V1;**
- дали не усложнява архитектурата излишно;
- и ако има по-добър вариант — ще го сравним с него.

И най-важното: **няма просто да ти кажа „да, прави това“**. Ще обсъдим защо.

Пращай го. 🙌

3. **Вие казахте:**

```
това е просто пример: [Range( StudentConstants.MinAge,
StudentConstants.MaxAge, ErrorMessage =
ValidationMessages.AgeMustBeBetweenMinAndMax)] public int Age {
get; set; } public static class ValidationMessages { public const string
AgeMustBeBetweenMinAndMax = "Age must be between {1} and
{2}."; }
```

4. **ChatGPT каза:**

Да — разбирам идеята ти. И всъщност **примерът ти е много близо до естественото решение**, но има един важен проблем.

Ти предлагаш:

```
[Range (
    StudentConstants.MinAge,
    StudentConstants.MaxAge,
    ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax) ]
public int Age { get; set; }
```

и:

```
public static class ValidationMessages
{
    public const string AgeMustBeBetweenMinAndMax =
        "Age must be between {1} and {2}.";
}
```

```
}
```

Проблемът е {1} и {2}

Тук има една коварна подробност.

При RangeAttribute:

```
[Range(minimum, maximum, ErrorMessage = "...")]
```

{1} и {2} **не означават** minimum и maximum по начина, по който логично бихме очаквали.

Затова при предишния ни тест получихме нещо от рода на:

```
Age must be between Age and 18.
```

Тоест DataAnnotations обработва placeholders-ите според собствената си логика, а не като:

```
{1} = StudentConstants.MinAge
```

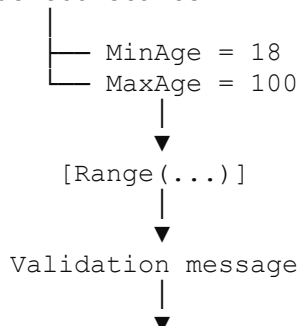
```
{2} = StudentConstants.MaxAge
```

Следователно **не бих използвал този вариант**.

Но идеята зад него е добра

Ти всъщност се опитваш да постигнеш нещо напълно разумно:

```
StudentConstants
```



```
"Age must be between 18 and 100."
```

Тоест **едно място да бъде source of truth** за 18 и 100.

Това според мен е правилното желание.

Въпросът е **как да го направим**, без да започнем да пишем собствен validation framework заради едно съобщение. 😊

Моето мнение за конкретния пример

Вариант	Оценка
"Age must be between 18 and 100."	✓ прост, но има дублиране
"Age must be between {1} and {2}."	✗ не работи както искаме
Custom ValidationAttribute	○ възможно, но вероятно прекалено за V1
Друг стандартен DataAnnotations подход	○ заслужава да го разгледаме

Аз **не бих бързал с custom attribute**.

Преди да пишем какъвто и да е код, бих ти показал един стандартен C#/.NET подход, който може да реши точно този проблем, и после ще го сравним с това, което ти си намерил. Ако искаш, **следващата ни стъпка може да е точно това — да разгледаме най-чистия стандартен вариант и защо работи**, без още да променяме проекта.

5. Вие казахте:

да пробваме твоите варианти

6. ChatGPT каза:

Да. 👍 Нека го направим като експеримент и **без да усложняваме проекта**.

Първо искам да коригирам едно важно нещо от предишния ми отговор: при RangeAttribute няма стандартен механизъм, при КОЙТО ErrorMessage = "Age must be between {1} and {2}." автоматично да вземе MinAge и MaxAge. Затова няма да се опитваме да „излъжем“ [Range].

Вариант 1 — най-простият

Оставяме:

```
[Range (
    StudentConstants.MinAge,
    StudentConstants.MaxAge,
    ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax) ]
public int Age { get; set; }
```

а съобщението:

```
public const string AgeMustBeBetweenMinAndMax =
    "Age must be between 18 and 100.";
```

Плюс

Много е ясно и напълно стандартно.

Минус

Имаме:

```
StudentConstants → 18, 100
ValidationMessages → 18, 100
```

Т.е. дублиране.

За V1 това всъщност е напълно приемливо, ако няма разумен начин да го избегнем без допълнителна сложност.

Вариант 2 — да генерираме съобщението от константите

Тук идеята е:

```
StudentConstants.MinAge  
StudentConstants.MaxAge
```

да са единственият source of truth.

Например концептуално:

```
"Age must be between " +  
StudentConstants.MinAge +  
" and " +  
StudentConstants.MaxAge +  
".";
```

Проблемът е, че ErrorMessage в [Range] трябва да бъде compile-time constant, а не произволно изчислена стойност.

Затова не можем просто да направим:

```
public static string AgeMustBeBetweenMinAndMax =>  
    $"Age must be between {StudentConstants.MinAge} and  
    {StudentConstants.MaxAge}.";
```

и да го подадем на attribute-a.

Вариант 3 — custom validation attribute

Това вече може да реши проблема напълно.

Например бихме могли да имаме нещо от типа:

```
[AgeRange]  
public int Age { get; set; }
```

и самият attribute да използва:

```
StudentConstants.MinAge  
StudentConstants.MaxAge
```

и да генерира:

```
Age must be between 18 and 100.
```

Това е технически чисто.

Но тук идва моето „но“.

За нашия проект ще създадем:

```
Custom Attribute
```

↓

```
Validation logic
```

↓

```
StudentConstants
```

↓

```
Validation message
```

само за да избегнем две числа в едно текстово съобщение.

За V1 според мен това е **overengineering**.

Вариант 4 — това е този, който бих предпочел да разгледаме

Можем да оставим [Range] да се занимава с диапазона:

```
[Range (
```

```
StudentConstants.MinAge,  
StudentConstants.MaxAge)]  
public int Age { get; set; }
```

а самото съобщение да бъде конфигурирано отделно.

Така разделяме двете отговорности:

```
[Range]
```

```
└─ Проверява:  $18 \leq \text{Age} \leq 100$ 
```

```
ValidationMessages
```

```
└─ Казва на потребителя какъв е проблемът
```

Но има един въпрос: **искаме ли изобщо персонализирано съобщение?**

Ако използваме стандартния [Range] без ErrorMessage, ASP.NET Core/DataAnnotations може само да ни даде стандартното съобщение за диапазона.

Това премахва нашата необходимост да поддържаме:

```
ValidationMessages.AgeMustBeBetweenMinAndMax
```

И тук стигаме до моето предпочитание

Аз бих избрал **най-простото решение, което има смисъл:**

```
[Range(  
    StudentConstants.MinAge,  
    StudentConstants.MaxAge)]  
public int Age { get; set; }
```

и бих **махнал специалното** AgeMustBeBetweenMinAndMax, ако стандартното съобщение е достатъчно добро за V1.

Тогава:

```
StudentConstants  
└─ MinAge = 18  
└─ MaxAge = 100
```

```
CreateStudentViewModel  
└─ [Range(MinAge, MaxAge)]
```

Няма дублиране.

Няма custom attribute.

Няма helper.

Няма допълнителна abstraction.

И най-важното — **не усложняваме архитектурата заради козметичен проблем.**

Но преди да го направим

Искам да видя **точния текущ** CreateStudentViewModel.cs, защото checkpoint-ът казва, че там имаме и другите validation-и:

```
Required  
EmailAddress
```

```
StringLength  
Range
```

Прати ми целия файл и ще направим **само първата малка промяна**. После Build.

7. Вие казахте:

```
namespace StudentManagement.ViewModels; using  
StudentManagement.Common.Constants; using  
System.ComponentModel.DataAnnotations; using  
StudentManagement.Common.Validation; public class  
CreateStudentViewModel { [Required] public string FirstName { get;  
set; } = string.Empty; [Required] public string LastName { get; set; } =  
string.Empty; [Required] [EmailAddress] [StringLength(100)] public  
string Email { get; set; } = string.Empty;  
[Range(StudentConstants.MinAge, StudentConstants.MaxAge,  
ErrorMessage =  
ValidationMessages.AgeMustBeBetweenMinAndMax)] public int Age {  
get; set; } public string Course { get; set; } = string.Empty; }
```

8. ChatGPT каза:

Супер. Сега вече виждам целия контекст и можем да вземем решение спокойно.

В момента имаме:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge,  
    ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax)]  
public int Age { get; set; }
```

и отделно:

```
public const string AgeMustBeBetweenMinAndMax =  
    "Age must be between 18 and 100.";
```

Моето предложение за V1

Аз бих **премахнал персонализираното** ErrorMessage и бих оставил стандартния [Range]:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]  
public int Age { get; set; }
```

Това е единствената промяна, която бих направил на този етап.

След това ValidationMessages.cs вече няма да

съдържа AgeMustBeBetweenMinAndMax, ако няма други съобщения в него.

Защо предпочитам това?

Защото реалната ни бизнес логика остава точно там, където трябва:

```

StudentConstants
├── MinAge = 18
└── MaxAge = 100
    ↓
CreateStudentViewModel
    ↓
[Range(18, 100)]

```

А [Range] си върши стандартната работа — проверява дали:
 $18 \leq \text{Age} \leq 100$

Не ни трябва:

- custom validation attribute;
- helper;
- dynamic property;
- специална логика за генериране на съобщението;
- дублиране на 18 и 100.

И това е много подходящо за нашето правило „**не правим abstraction само заради самия pattern**“.

Единственото нещо, което губим

Губим нашето конкретно съобщение:

Age must be between 18 and 100.

и ще използваме стандартното съобщение на DataAnnotations.

За **V1** аз смятам, че това е напълно приемлив компромис.

Предлагам първата малка промяна

В CreateStudentViewModel.cs замени само това:

```

[Range(StudentConstants.MinAge, StudentConstants.MaxAge,
    ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax)]

```

с:

```

[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]

```

Нищо друго не променяй засега.

После направи **Build** и ми кажи резултата:

Build: ?

Errors: ?

Warnings: ?

След Build ще решим какво правим с ValidationMessages.cs. Не искам да изтриваме нищо предварително.

9. Вие казахте:

може ли да заменим/override стандартното съобщение с наше

10. ChatGPT каза:

Да, **може** — и всъщност това променя малко предишната ми препоръка. 😊

Можем да запазим стандартния [Range], но да му дадем **наше съобщение**. Въпросът е как да избегнем дублирането на 18 и 100.

И тук има един доста чист вариант: **да не слагаме числата в ValidationMessages, а да използваме placeholders, които се попълват от RangeAttribute**. Но трябва да внимаваме какви placeholders поддържа конкретно RangeAttribute.

По-добрият подход за нашия случай е да използваме стандартното съобщение на RangeAttribute като основа, вместо да създаваме custom attribute.

Ако държим на точно:

Age must be between 18 and 100.

и искаме 18 и 100 да идват автоматично от:

StudentConstants.MinAge

StudentConstants.MaxAge

тогава вече си заслужава да разгледаме **как ASP.NET Core локализира/форматира validation messages**.

Но преди да пишем код, бих разделил въпроса на две:

1. **Можем ли да override-нем стандартното съобщение?**
Да — абсолютно.
2. **Можем ли едновременно да имаме наше съобщение и автоматично да вкараме MinAge/MaxAge в него само със стандартния [Range]?**

Тук е уловката и точно това трябва да решим.

И аз бих искал да го направим **без custom attribute**, ако има стандартен начин.

Ако искаш, можем сега да разгледаме конкретно **какво реално поддържа RangeAttribute.ErrorMessage и защо {1} / {2} дадоха онзи странен резултат**, след което ще изберем решение.

11. Вие казахте:

ОК

12. ChatGPT каза:

ОК. Нека първо изясним **точно какво ни позволява [Range]**, без да променяме проекта.

Какво става с ErrorMessage

Имаме:


```
[Range (
    StudentConstants.MinAge,
    StudentConstants.MaxAge,
    ErrorMessage = "Age must be between {1} and {2}.")]
```

Изглежда логично да очакваме:

```
{1} → 18
{2} → 100
```

Но `ValidationAttribute` ИЗПОЛЗВА `FormatErrorMessage()` и конкретният `RangeAttribute` подава свои аргументи към форматирането. Затова **не можем да разчитаме {1} и {2} да са min/max.**

Точно това обяснява защо предишният тест даде:

Age must be between Age and 18.

Тоест идеята ти е добра, но placeholder-ите не са правилният механизъм за това.

А можем ли да override-нем съобщението?

Да:

```
[Range (
    StudentConstants.MinAge,
    StudentConstants.MaxAge,
    ErrorMessage = "Age must be between 18 and 100.")]
```

Това работи.

Но отново имаме:

```
StudentConstants → 18, 100
ErrorMessage     → 18, 100
```

Има още един вариант, който според мен си заслужава

Вместо да държим съобщението като `const`, можем да го генерираме **при изпълнение**, използвайки `StudentConstants`.

Например идеята е:

```
 $"Age must be between {StudentConstants.MinAge} and {StudentConstants.MaxAge}."
```

Това решава дублирането.

Проблемът е, че не можем просто да сложим такава runtime стойност в:

```
ErrorMessage = ...
```

защото `argument`-ът на `attribute` трябва да е `compile-time constant`.

Затова тук вече имаме реална архитектурна граница на стандартния `[Range]`.

Моето заключение

За нашия проект бих подредил вариантите така:

1. 🏆 **Custom validation attribute** — технически най-гъвкав, но прекалено много код за една проста проверка.
2. 🥈 **Стандартен [Range] + наше статично съобщение** — най-просто и напълно нормално за V1.
3. 🥉 **Стандартен [Range] без наше съобщение** — най-просто, но губим желаното UX съобщение.

И понеже ти искаш **да имаме наше съобщение**, аз бих избрал **№2 за V1**.

Тоест:

```
[Range (
    StudentConstants.MinAge,
    StudentConstants.MaxAge,
    ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax) ]
```

и:

```
public const string AgeMustBeBetweenMinAndMax =
    "Age must be between 18 and 100.";
```

Това **не е перфектно откъм DRY**, но е просто, ясно и надеждно. Ако по-късно имаме още validation-и, които изискват динамични стойности, тогава вече ще имаме **реална причина** да въведем по-сериозно решение.

Аз бих предпочел това пред custom attribute само заради 18 и 100.

1. ако кажа да, по какъв начин бихме показали какъв е диапазона
2. **ChatGPT каза:**

Да — ако под „да“ имаш предвид **да запазим нашето съобщение**, но да не правим custom attribute, имаме няколко начина да покажем диапазона.

Вариант А — директно в съобщението

```
public const string AgeMustBeBetweenMinAndMax =
    "Age must be between 18 and 100.";
```

При Age = 17 потребителят вижда:

Age must be between 18 and 100.

Това е най-простият вариант.

Вариант В — показваме диапазона и в самото поле

Например във формата:

```
Age
[ 17 ]
```

Age must be between 18 and 100.

или още по-добре:

```
Age
[ 17 ]
Enter an age between 18 and 100.
```

Тук можем да използваме `StudentConstants.MinAge` и `StudentConstants.MaxAge` за текста на страницата, вместо да ги дублираме във `validation message`.

Вариант С — HTML min / max

Това ми харесва още повече за нашия `Create form`.

Можем да имаме:

```
<input type="number" min="18" max="100">
```

Така браузърът знае:

Минимум: 18

Максимум: 100

А сървърната валидация си остава:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

Това ни дава **две нива на защита**:

Browser

↓

```
min="18" max="100"
```

↓

по-добър UX

+

ASP.NET Core

↓

```
[Range(18, 100)]
```

↓

server-side validation

И тук има важен момент: **HTML validation не замества server-side validation**. Потребителят може да заобиколи browser validation, затова `[Range]` си остава задължителен.

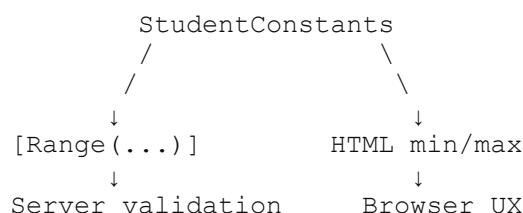
Какво бих избрал аз?

За нашия проект бих направил **С + наше разбираемо съобщение**.

Тоест потребителят вижда диапазона още във формата, а ако въведе грешна стойност, получава ясно съобщение.

И тогава въпросът с дублирането на 18 и 100 става много по-интересен — можем да направим така, че **самите HTML min/max да идват от StudentConstants**, докато `[Range]` също използва същите константи.

Така:



Това според мен е по-смислена посока за проекта, отколкото да създаваме custom attribute само за да генерираме едно съобщение.

3. Вие казахте:

не ме разбра

4. ChatGPT каза:

Да, разбрах те неправилно. 👍

Ти питаш по-конкретно:

Ако кажем „да“ на вариант с наше съобщение, но не искаме да пишем 18 и 100 ръчно в ValidationMessages, тогава по какъв начин самото съобщение ще покаже диапазона?

Тоест искаш нещо като:

Age must be between 18 and 100.

но 18 и 100 да бъдат взети автоматично от:

StudentConstants.MinAge

StudentConstants.MaxAge

а не да ги пишем втори път.

Да — точно това е въпросът.

И тук трябва да поправа предишното си обяснение: не искам да те водя към решение, което само изглежда DRY, но реално не работи.

Ако това е въпросът ти, нека разгледаме **конкретно как можем да генерираме съобщението от MinAge и MaxAge и едновременно да го използваме при [Range]**. Това вече е правилният проблем за решаване.

5. Вие казахте:

за запазим Range без ErrorMessage, но да покажем диапазона дори като информативно

6. ChatGPT каза:

Ааа, сега те разбрах. 👍

Ти предлагаш:

Да махнем ErrorMessage от [Range], да оставим стандартната валидация, но във формата предварително/информативно да покажем допустимия диапазон.

И да — това според мен е много добър вариант за V1.

Как бих го направил

Оставяме ViewModel-а максимално чист:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]  
public int Age { get; set; }
```

Така [Range] продължава да валидира от:

18 → 100

А в Create.cshtml можем да покажем например:

```
Age  
[      ]
```

Allowed age: 18-100

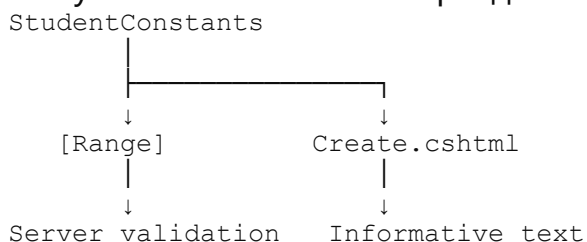
Още по-добре, можем да направим самото поле number с HTML ограничения:

```
Age  
[      ]  
Must be between 18 and 100.
```

Тогава потребителят **знае диапазона още преди да въведе грешна стойност.**

Има още един плюс

Получаваме много чисто разделение:



[Range] казва:

„Това е правилото.“

А интерфейсът казва:

„Ето какво се очаква от теб.“

И няма нужда ValidationMessages да знае за 18 и 100.

Единственото, което трябва да решим

Ако напишем в Create.cshtml:

Must be between 18 and 100.

пак ще имаме дублиране.

Затова следващият интересен въпрос е **как да**

покажем StudentConstants.MinAge и MaxAge **в Razor view**, без да копираме числата.

И това вече е напълно нормална задача за Razor — **не е нужно да правим custom validation.**

Аз бих продължил точно натам.

1. не ме разбра

2. ChatGPT каза:

Да, разбрах те неправилно. 👍

Ти питаш по-конкретно:

Ако кажем „да“ на вариант с наше съобщение, но не искаме да пишем 18 и 100 ръчно в ValidationMessages, тогава по какъв начин самото съобщение ще покаже диапазона?

Тоест искаш нещо като:

Age must be between 18 and 100.

но 18 и 100 да бъдат взети автоматично от:

StudentConstants.MinAge

StudentConstants.MaxAge

а не да ги пишем втори път.

Да — точно това е въпросът.

И тук трябва да поправа предишното си обяснение: не искам да те водя към решение, което само изглежда DRY, но реално не работи.

Ако това е въпросът ти, нека разгледаме **конкретно как можем да генерираме съобщението от MinAge и MaxAge и едновременно да го използваме при [Range]**. Това вече е правилният проблем за решаване.

3. Вие казахте:

за запазим Range без ErrorMessage, но да покажем диапазона дори като информативно

4. ChatGPT каза:

Ааа, сега те разбрах. 👍

Ти предлагаш:

Да махнем ErrorMessage от [Range], да оставим стандартната валидация, но във формата предварително/информативно да покажем допустимия диапазон.

И да — това според мен е много добър вариант за V1.

Как бих го направил

Оставяме ViewModel-а максимално чист:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]  
public int Age { get; set; }
```

Така [Range] продължава да валидира от:

18 → 100

А в Create.cshtml можем да покажем например:

```
Age
[      ]
```

Allowed age: 18-100

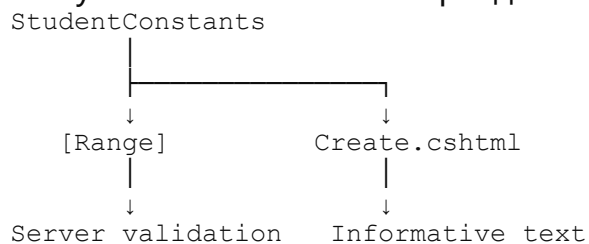
Още по-добре, можем да направим самото поле `number` с HTML ограничения:

```
Age
[      ]
Must be between 18 and 100.
```

Тогава потребителят **знае диапазона още преди да въведе грешна стойност.**

Има още един плюс

Получаваме много чисто разделение:



`[Range]` казва:

„Това е правилото.“

А интерфейсът казва:

„Ето какво се очаква от теб.“

И няма нужда `ValidationMessages` да знае за 18 и 100.

Единственото, което трябва да решим

Ако напишем в `Create.cshtml`:

```
Must be between 18 and 100.
```

пак ще имаме дублиране.

Затова следващият интересен въпрос е **как да покажем** `StudentConstants.MinAge` и `MaxAge` **в Razor view**, без да копираме числата.

И това вече е напълно нормална задача за Razor — **не е нужно да правим custom validation.**

Аз бих продължил точно натам.

5. Вие казахте:

Must be between 18 and 100. -> "Age->must be between 18 and 100"

6. ChatGPT каза:

Да — точно. 😊

Идеята е информативният текст да бъде например:

Age → must be between 18 and 100.

или визуално във формата:

```
Age
[      ]
```

Age → must be between 18 and 100.

При това **не използваме** ErrorMessage **за диапазона**.

В ViewModel остава само:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
public int Age { get; set; }
```

А в Create.cshtml показваме информативния текст.

И най-важното — можем да направим 18 и 100 да идват от:

```
StudentConstants.MinAge
StudentConstants.MaxAge
```

така че да няма дублиране.

Това според мен е **по-добър UX вариант** от това да чакаме потребителят да въведе 17, за да разбере какъв е диапазонът.

Получаваме:

Информация за потребителя

↓

Age → must be between 18 and 100.

+

Server-side validation

↓

```
[Range(MinAge, MaxAge)]
```

И бих избрал този подход за V1. **Не пиаме още**

кода. Следващата малка стъпка е да видим текущия Create.cshtml и да добавим този текст по правилния Razor начин.

7. Вие казахте:

или при фокус на полето да се показва

8. ChatGPT каза:

Да — **това ми харесва повече**, особено ако искаме формата да остане чиста.

Идеята:

```
Age
[      ]
      ↑
    потребителят
    натиска полето
```

Age → must be between 18 and 100.

Age

[]

Защо ми харесва

Focus върху Age

↓
Информация

Age = 17

↓
X стандартното validation съобщение

И тук JavaScript има реален смисъл

focus

blur

И можем да използваме `StudentConstants.MinAge` и `MaxAge`, така че да не пишем 18 и 100 ръчно.

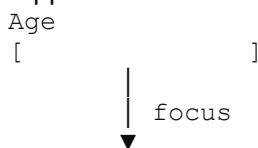
После, ако искаме по-контролирано поведение, добавяме малко JS.

Аз лично бих пробвал първо HTML/CSS варианта. Ако не ни даде точното поведение, което искаме — тогава JS.

без JS

Да. **Без JS** — и според мен това е по-добрият избор за момента. Можем да направим поведението само с **CSS**, използвайки `:focus`.

Идеята е:



i Age → must be between 18 and 100.

Когато полето не е на фокус:

```
Age
[
  ]
```

Как ще го направим

В Create.cshtml ще имаме hint елемент:

```
<div class="age-hint">
  Age → must be between 18 and 100.
</div>
```

и чрез CSS:

```
.age-hint {
  display: none;
}

#Age:focus + .age-hint {
  display: block;
}
```

Но: това е само пример за механизма. Не искам още да го копираш, защото трябва да видя как точно е структурирано твоето текущо Create.cshtml.

После можем да направим и по-чистото:

```
StudentConstants
  ↓
Create.cshtml
  ↓
18 / 100
  ↓
Age → must be between 18 and 100.
```

Така няма да пишем 18 и 100 ръчно в HTML.

И [Range] остава:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

Така получаваме точно това, което искаш:

- **без JS;**
- стандартен [Range];
- без custom validation;
- диапазонът е видим **само когато полето е във фокус;**
- 18 и 100 имат едно основно място — StudentConstants.

Следващата стъпка: изпрати ми актуалния Create.cshtml и ще направим само тази малка промяна.

1. `<div class="age-hint"> Age → must be between 18 and 100. </div>`
само кажи числата да ги взима от StudentConstants
2. **ChatGPT каза:**

Да. В Razor можеш директно да използваш константите:

```
<div class="age-hint">
    Age → must be between @StudentConstants.MinAge and
    @StudentConstants.MaxAge.
</div>
```

Така:

```
StudentConstants
├─ MinAge = 18
└─ MaxAge = 100
```

↓
Create.cshtml

↓
Age → must be between 18 and 100.

И ако утре променим MinAge на 16, текстът автоматично ще стане:

Age → must be between 16 and 100.

Само провери дали Create.cshtml **има достъп до** StudentConstants **чрез:**

@using StudentManagement.Common.Constants

или ако използваш fully qualified name.

Това е чисто решение и **не ни трябва JS**.

3. **Вие казахте:**

и ми кажи как да добавя и min и max на input value

4. **ChatGPT каза:**

Да. Ако искаме и самият `<input>` да получи min и max от StudentConstants, можем да направим:

```
<input asp-for="Age"
       type="number"
       min="@StudentConstants.MinAge"
       max="@StudentConstants.MaxAge" />
```

Така HTML ще получи реално:

```
<input type="number" min="18" max="100">
```

А когато променим константите, например:

```
MinAge = 16;
```

```
MaxAge = 90;
```

Razor автоматично ще генерира:

```
<input type="number" min="16" max="90">
```

Така имаме **едно място за диапазона**:

```
StudentConstants
├─ [Range (MinAge, MaxAge)]
├─ input min="MinAge"
└─ input max="MaxAge"
```

└─ Age hint

↓
"Age → must be between 18 and 100."

Това вече ми харесва като решение за V1. Чисто е и не ни трябва нито JS, нито custom validation.

5. Вие казахте:

дори някои да се опита нещо да промени си остава защита ни на ниво DB

6. ChatGPT каза:

Точно — **но с едно важно уточнение**: min И max в HTML **не са защита**. Те са UX/browser validation.

Истинската защита трябва да е на сървъра:

Browser

↓

min="18" / max="100"

↓

UX + client-side validation

↓

ASP.NET Core

↓

[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]

↓

Service

↓

Database

Ако някой промени HTML през DevTools например:

min="1" max="200"

това **не заобикаля** нашия [Range].

А за DB — да, това е още едно ниво

Идеалната защита е:

HTML

↓

ASP.NET Core validation

↓

Service/business rules

↓

Database constraints

Така не разчитаме на клиента.

Само че трябва да проверим как е дефиниран Age в Student и каква е текущата ни SQL Server база, преди да кажем, че **в момента** DB действително налага 18–100.

Ако DB няма CHECK CONSTRAINT, например, тогава базата сама по себе си няма да спре Age = 5.

Така че аз бих казал:

HTML min/max → **UX**

[Range] → **server-side validation**

DB constraint → **database-level защита**

И трите имат различна роля.

7. Вие казахте:

знам и точно затова го казах

8. ChatGPT каза:

Да 😊, разбрах те — ти точно това имаше предвид.

И всъщност мислиш в правилната посока:

min / max в HTML

↓

помага на нормалния потребител

[Range]

↓

server-side защита

Database constraint

↓

последна линия на защита

Тоест дори някой нарочно да промени HTML-а, **не разчитаме на него за сигурността.**

И ми харесва решението, до което стигнахме за Age:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

```
public int Age { get; set; }
```

```
<input asp-for="Age"
```

```
    type="number"
```

```
    min="@StudentConstants.MinAge"
```

```
    max="@StudentConstants.MaxAge" />
```

и информативно:

```
<div class="age-hint">
```

```
    Age → must be between @StudentConstants.MinAge and
```

```
@StudentConstants.MaxAge.
```

```
</div>
```

Една двойка константи → три места, които използват същите стойности.

Без JS, без custom attribute и без дублиране на 18 и 100.

Това вече е доста чисто решение за V1. 👍

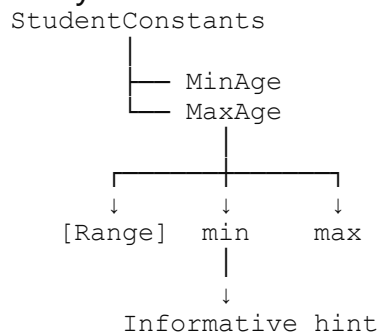
9. Вие казахте:

намерихне доста чисто и просто решение

10. ChatGPT каза:

Да! 😊 И според мен това е точно типът решение, който търсихме — **просто, четимо и без излишна архитектура.**

Получихме:



И най-хубавото:

- няма дублиране на 18 и 100;
- няма custom validation attribute;
- няма JavaScript;
- `[Range]` си остава стандартният server-side validation;
- HTML min/max подобрява UX;
- hint-ът показва диапазона информативно;
- при промяна на константите всичко се актуализира автоматично.

Това според мен е **много по-добро за нашия V1**, отколкото да усложняваме нещата само заради персонализирано validation съобщение.

Следващото логично нещо е да го имплементираме **малко по малко** и да направим Build.

11. Вие казахте:

да го направим и за останалите полета

12. ChatGPT каза:

Да — но тук бих направил **една важна уговорка**: не всички полета имат смисъл да получат min/max.

Имаме:

FirstName
LastName
Email
Age
Course

За Age вече имаме реален числов диапазон.

За останалите можем да използваме подходящите ограничения, които **вече имаме в validation-a**, вместо да измисляме нови.

Например:

FirstName



[Required]

LastName



[Required]

Email



[Required]

[EmailAddress]

[StringLength(100)]

Age



[Range(18, 100)]

min="18"

max="100"

Course



няма конкретен validation диапазон в момента

За Email например

Тук вече имаме:

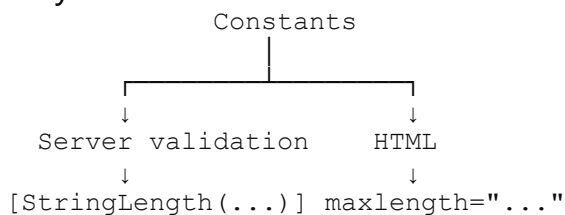
[StringLength(100)]

и можем да дадем на HTML input-a:

maxlength="@..."

Но **не бих писал** maxlength="100" **директно**. Ако искаме същия принцип като при Age, първо можем да изнесем 100 в подходяща константа и после и DataAnnotations, и HTML да я използват.

Получаваме:



Това е същата идея, която използвахме при Age.

Но да го направим по правилото ни — едно поле наведнъж.

Предлагам първо да завършим **Age** докрай в Create.cshtml: min, max + hint при focus, без JS. После Build и чак тогава минаваме на Email.

13. Вие казахте:

ние намерихме решение за това поле как да се валидира да го имплементираме и да валидираме и останалите полета спаред това което се изисква

1. Вие казахте:

ние намерихме решение за това поле как да се валидира да го имплементираме и да валидираме и останалите полета спаред това което се изисква

2. ChatGPT каза:

Да. Сега вече сме на една ясна посока. 👍

Идеята е не просто да „украсим“ полетата, а да направим **цялата Create форма последователна** — всеки validation rule да има съответното HTML поведение, когато има смисъл.

Ще го направим постепенно.

1. Age — първо имплементираме намереното решение

В CreateStudentViewModel.cs:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
public int Age { get; set; }
```

Махаме ErrorMessage, защото вече няма да го използваме.

В Create.cshtml:

```
<input asp-for="Age"
       type="number"
       min="@StudentConstants.MinAge"
       max="@StudentConstants.MaxAge" />
```

```
<div class="age-hint">
    Age → must be between @StudentConstants.MinAge and
    @StudentConstants.MaxAge.
</div>
```

След това CSS ще се погрижи hint-ът да се показва при focus, без JavaScript.

Но **преди да правим CSS**, първо искам да завършим validation моделите.

2. FirstName

В момента:

```
[Required]
public string FirstName { get; set; } = string.Empty;
```

Required означава, че не трябва да бъде празно.

Тук бихме могли да добавим и разумен StringLength, но **не трябва да измисляме произволно максимална дължина**.

Ако в Student модела вече има например:

```
[StringLength(50)]
public string FirstName { get; set; }
```

тогава можем да използваме същото ограничение и във ViewModel-a.

3. LastName

Същото:

```
[Required]
```

```
public string LastName { get; set; } = string.Empty;
```

Първо проверяваме какво изисква Student моделът.

4. Email

Тук вече имаме конкретни правила:

```
[Required]
```

```
[EmailAddress]
```

```
[StringLength(100)]
```

```
public string Email { get; set; } = string.Empty;
```

Това е доста добре.

И можем да синхронизираме HTML:

```
maxlength="100"
```

но, както при Age, **не искам да пишем 100 втори път.**

По-чисто ще бъде:

```
StudentConstants
```

```
└─ MaxEmailLength = 100
```

```
└─ [StringLength(MaxEmailLength)]
```

```
└─ maxlength="@StudentConstants.MaxEmailLength"
```

Това е същият принцип.

5. Course

В момента:

```
public string Course { get; set; } = string.Empty;
```

Тук няма validation.

Преди да добавяме Required, трябва да решим дали **Course** **действително е задължително поле според изискванията на проекта.**

Не искам да добавяме validation само защото „така е по-добре“.

Така бих подредил работата

```
CreateStudentViewModel
```

```
└─ FirstName → Required + реалните ограничения
```

```
└─ LastName → Required + реалните ограничения
```

```
└─ Email → Required + EmailAddress + StringLength
```

```
└─ Age → Range + HTML min/max + hint
```

```
└─ Course → според изискванията
```

И след това:

```
ViewModel validation
↓
HTML validation / UX
↓
POST
↓
ModelState
↓
Mapping
↓
Service
↓
Database
```

Но сега нека не гадаем.

За да валидираме „според това, което се изисква“, изпрати ми актуалния:

Models/Student.cs

и ако имаме конкретни изисквания за полетата (например от заданието на проекта) — и тях.

Тогава ще определим **точните validation правила за всяко поле**, вместо да измисляме ограничения. След това ги имплементираме едно по едно и правим Build след всяка логична стъпка.

3. Вие казахте:

може ли стъпка по стъпка

4. ChatGPT каза:

Да. Точно така ще го направим — **една стъпка наведнъж**. 👍

Стъпка 1 — Age

Променяме **само** Age в CreateStudentViewModel.cs.

В момента е:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge,
    ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax)]
public int Age { get; set; }
```

Замени го с:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
public int Age { get; set; }
```

Това е **единствената промяна засега**.

След това направи **Build**.

Кажи ми само резултата:

```
Build:
Errors:
Warnings:
```

После продължаваме със следващата малка стъпка.

5. Вие казахте:

```
Build started at 9:23 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 9:23 PM and took 05.942 seconds =====
```

6. ChatGPT каза:

Перфектно. 

```
Build: 1 succeeded
Errors: 0
Warnings: 0
```

Age вече използва само:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

и махнахме зависимостта му от ValidationMessages.

Стъпка 2 — махаме вече ненужното съобщение

Сега **само една малка промяна.**

В Common/Validation/ValidationMessages.cs в момента имаме:

```
public static class ValidationMessages
{
    public const string AgeMustBeBetweenMinAndMax =
        "Age must be between 18 and 100.";
}
```

Понеже вече не използваме това съобщение, **изтрий само**

константата и остави класа така:

```
namespace StudentManagement.Common.Validation;
```

```
public static class ValidationMessages
{
}
```

Не променяй нищо друго.

После пак **Build.**

Кажи ми резултата и продължаваме.

7. Вие казахте:

```
Build started at 9:24 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
```

failed, 0 up-to-date, 0 skipped ===== Build
completed at 9:24 PM and took 00.694 seconds =====

8. ChatGPT каза:

Перфектно. 

Build: 1 succeeded

Errors: 0

Warnings: 0

Имаме чисто състояние.

Стъпка 3 — почистваме CreateStudentViewModel

В началото на файла все още имаме:

```
using StudentManagement.Common.Validation;
```

Това вече **не е необходимо**, защото ValidationMessages не се

използва никъде в CreateStudentViewModel.

Изтрий само този ред:

```
using StudentManagement.Common.Validation;
```

Останалото **не го пипай**.

След това:

Build → **изпрати резултата**.

После ще преминем към Create.cshtml и ще добавим min, max и информативния текст за Age.

9. Вие казахте:

Build started at 9:25 PM... 1>----- Build started: Project:

StudentManagement, Configuration: Debug Any CPU -----

1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net

8.0\StudentManagement.dll ===== Build: 1 succeeded, 0

failed, 0 up-to-date, 0 skipped ===== Build

completed at 9:25 PM and took 00.707 seconds =====

10. ChatGPT каза:

Перфектно. 

Build: 1 succeeded

Errors: 0

Warnings: 0

Age validation вече е чисто отделена от ValidationMessages.

Стъпка 4 — Create.cshtml

Сега ще направим **само Age**.

Искам да видя актуалния Create.cshtml, преди да променяме нещо.

Изпрати ми целия файл:

Views/Students/Create.cshtml

После ще добавим само:

- type="number"
- min="@StudentConstants.MinAge"
- max="@StudentConstants.MaxAge"
- информативния текст:
Age → must be between 18 and 100.

и ще го направим така, че **18 и 100 да идват от** StudentConstants.

След това ще се погрижим за показването при focus **без JavaScript**.

1. @model StudentManagement.ViewModels.CreateStudentViewModel
<h1>Add Student</h1> <form asp-action="Create"
method="post"> <div class="mb-3"> <label asp-for="FirstName"
class="form-label"> </label> <input asp-for="FirstName"
class="form-control" /> <span asp-validation-for="FirstName"
class="text-danger"> </div> <div class="mb-3"> <label
asp-for="LastName" class="form-label"> </label> <input asp-
for="LastName" class="form-control" /> <span asp-validation-
for="LastName" class="text-danger"> </div> <div
class="mb-3"> <label asp-for="Email" class="form-label"> </label>
<input asp-for="Email" class="form-control" /> <span asp-
validation-for="Email" class="text-danger"> </div> <div
class="mb-3"> <label asp-for="Course" class="form-label"> </label>
<input asp-for="Course" class="form-control" /> <span asp-
validation-for="Course" class="text-danger"> </div> <div
class="mb-3"> <label asp-for="Age" class="form-label"> </label>
<input asp-for="Age" class="form-control" /> <span asp-validation-
for="Age" class="text-danger"> </div> <button
type="submit" class="btn btn-primary">Save</button> <a asp-
action="Index" class="btn btn-secondary">Cancel </form>
2. **ChatGPT каза:**

Да, виждам го. 🙌 Ще променим **само Age**, без да пипаме останалите полета.

Първо ни трябва достъп до StudentConstants в Razor файла. Добави най-отгоре, **точно под** @model:

@using StudentManagement.Common.Constants

Така началото ще стане:

```
@model StudentManagement.ViewModels.CreateStudentViewModel
@using StudentManagement.Common.Constants
```

Само това засега.

Не променяй Age input-а още.

След това направи **Build** и ми кажи резултата.

3. Вие казахте:

```
Build started at 9:28 PM... 1>----- Build started: Project:
StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net
8.0\StudentManagement.dll ===== Build: 1 succeeded, 0
failed, 0 up-to-date, 0 skipped ===== Build
completed at 9:28 PM and took 00.703 seconds =====
```

4. ChatGPT каза:

Перфектно. 

```
Build: 1 succeeded
Errors: 0
Warnings: 0
```

Стъпка 5 — добавяме min и max на Age

Сега променяме **само input-а за Age**.

В момента:

```
<input asp-for="Age" class="form-control" />
```

Замени го с:

```
<input asp-for="Age"
      type="number"
      min="@StudentConstants.MinAge"
      max="@StudentConstants.MaxAge"
      class="form-control" />
```

Това ще генерира приблизително:

```
<input type="number" min="18" max="100">
```

но стойностите идват от StudentConstants, а не са написани ръчно.

Не добавяй hint-а още. Само това.

После направи Build и ми дай резултата.

1. Build started at 9:30 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:30 PM and took 00.686 seconds =====

2. ChatGPT каза:

Перфектно. 

Build: 1 succeeded

Errors: 0

Warnings: 0

Сега Age има едновременно:

- [Range] → server-side validation
- min → browser/HTML ограничение
- max → browser/HTML ограничение

Стъпка 6 — добавяме информативния текст

Сега ще добавим **само hint-а**.

В Age блока имаш:

```
<label asp-for="Age" class="form-label"></label>
<input asp-for="Age"
      type="number"
      min="@StudentConstants.MinAge"
      max="@StudentConstants.MaxAge"
      class="form-control" />
<span asp-validation-for="Age" class="text-danger"></span>
```

Между input и span добави:

```
<div class="age-hint">
    Age → must be between @StudentConstants.MinAge and
    @StudentConstants.MaxAge.
</div>
```

Така целият блок ще бъде:

```
<div class="mb-3">
    <label asp-for="Age" class="form-label"></label>
    <input asp-for="Age"
          type="number"
          min="@StudentConstants.MinAge"
          max="@StudentConstants.MaxAge"
          class="form-control" />

    <div class="age-hint">
        Age → must be between @StudentConstants.MinAge and
        @StudentConstants.MaxAge.
    </div>

    <span asp-validation-for="Age" class="text-danger"></span>
</div>
```

Все още няма CSS. Засега текстът ще се вижда постоянно. След Build ще направим следващата малка стъпка — да го скрием и да го показваме при focus, без JavaScript.

Направи само тази промяна → **Build** → изпрати резултата.

1. Build started at 9:31 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->

D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:31 PM and took 00.717 seconds =====

2. ChatGPT каза:

Супер. ☒

Build: 1 succeeded

Errors: 0

Warnings: 0

Age вече има и информативния текст, като диапазонът идва ОТ StudentConstants.

Стъпка 7 — скриваме hint-а

Сега ще направим **само CSS промяната**. Няма JavaScript.

Идеята е:

Полето не е във фокус



Age → must be between 18 and 100. ☒ скрито

Полето е във фокус



Age → must be between 18 and 100. ☒ показано

Преди да ти кажа **къде точно да сложиш CSS-а**, обаче, не искам да гадая структурата на проекта.

Изпрати ми съдържанието на:

Views/Shared/_Layout.cshtml

и ще видим къде е най-подходящото място за CSS в нашия проект.

1.

```
<!DOCTYPE html> <html lang="en"> <head> <meta charset="utf-8" /> <meta name="viewport" content="width=device-width, initial-scale=1.0" /> <title>@ViewData["Title"] - StudentManagement</title> <link rel="stylesheet" href="~/lib/bootstrap/dist/css/bootstrap.min.css" /> <link rel="stylesheet" href="~/css/site.css" asp-append-version="true" /> <link rel="stylesheet" href="~/StudentManagement.styles.css" asp-append-version="true" /> </head> <body> <header> <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3"> <div class="container-fluid"> <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">StudentManagement</a> <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-
```



```

controls="navbarSupportedContent" aria-expanded="false" aria-
label="Toggle navigation"> <span class="navbar-toggler-
icon"> </span> </button> <div class="navbar-collapse collapse d-
sm-inline-flex justify-content-between"> <ul class="navbar-nav flex-
grow-1"> <li class="nav-item"> <a class="nav-link text-dark" asp-
area="" asp-controller="Home" asp-action="Index">Home</a> </li>
<li class="nav-item"> <a class="nav-link text-dark" asp-area="" asp-
controller="Students" asp-action="Index">Students</a> </li> <li
class="nav-item"> <a class="nav-link text-dark" asp-area="" asp-
controller="Home" asp-action="Privacy">Privacy</a> </li> </ul>
</div> </div> </nav> </header> <div class="container"> <main
role="main" class="pb-3"> @RenderBody() </main> </div> <footer
class="border-top footer text-muted"> <div class="container">
&copy; 2026 - StudentManagement - <a asp-area="" asp-
controller="Home" asp-action="Privacy">Privacy</a> </div>
</footer> <script src="~/lib/jquery/dist/jquery.min.js"> </script>
<script
src="~/lib/bootstrap/dist/js/bootstrap.bundle.min.js"> </script>
<script src="~/js/site.js" asp-append-version="true"> </script>
@await RenderSectionAsync("Scripts", required: false) </body>
</html>

```

2. ChatGPT каза:

Супер. Виждам, че вече имаме site.css, който се зарежда глобално:

```

<link rel="stylesheet" href="~/css/site.css" asp-append-
version="true" />

```

Това е правилното място за нашия малък CSS, защото age-hint е визуално поведение на формата, а не C# логика.

Стъпка 7 — само добавяме CSS

Отвори:

```
wwwroot/css/site.css
```

и **най-долу** добави:

```

.age-hint {
    display: none;
}

input[name="Age"]:focus + .age-hint {
    display: block;
}

```

Тук има една важна подробност: при нас структурата е:

```

<input ... />
<div class="age-hint">
    ...
</div>

```


затова:

```
input[name="Age"]:focus + .age-hint
```

означава:

„Намери input-а за Age, когато е във фокус, и покажи непосредствения div след него.“

Така:

```
Age input
```

```
  ↓ focus
```

```
.age-hint
```

```
  ↓
```

```
display: block
```

А когато Age не е във фокус:

```
.age-hint
```

```
  ↓
```

```
display: none
```

Само добави този CSS засега. Не променяй друго.

След това направи Build и ми кажи резултата.

1. Build started at 9:36 PM... 1>----- Build started: Project: StudentManagement, Configuration: Debug Any CPU -----
1>StudentManagement ->
D:\GitHub\StudentManagement\StudentManagement\bin\Debug\net 8.0\StudentManagement.dll ===== Build: 1 succeeded, 0 failed, 0 up-to-date, 0 skipped ===== Build completed at 9:36 PM and took 00.365 seconds =====
2. **ChatGPT каза:**

Перфектно. 

Build: 1 succeeded

Errors: 0

Warnings: 0

Сега правим проверка в браузъра — без нова промяна

Стартирай приложението и отвори **Add Student / Create**.

Провери само Age:

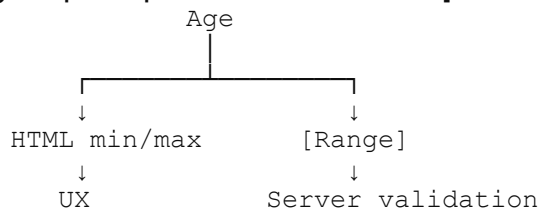
1. Когато страницата се зареди → hint-ът трябва да е **скрит**.
2. Кликни върху Age → трябва да се появи:

Age → must be between 18 and 100.

3. Кликни извън Age → hint-ът трябва да се скрие.
4. Въведи 17 и опитай да запишеш → трябва да получиш validation error от [Range].
5. Въведи 18 → трябва да е валидно.

- Въведи 100 → трябва да е валидно.
- Въведи 101 → трябва да е невалидно.

Тук проверяваме **и UX-а, и реалната validation логика**:



Кажете ми как се държи при тези тестове. **Не променяме нищо повече, докато не потвърдим Age.**

- не се появява hint
- ChatGPT каза:**

Да — намерих вероятната причина. 👍

Нашият CSS:

```
input[name="Age"]:focus + .age-hint
```

разчита `div.age-hint` да бъде **директно след input-а** в генерирания HTML. Това би трябвало да работи при текущата структура, но нека не гадаем.

Има и по-надежден CSS вариант, който е подходящ точно за нашия случай: `:focus-within`.

Но първо — **не променяй нищо**.

Стъпка 8 — проверяваме генерирания HTML

В браузъра отвори Create страницата:

- Натисни **F12**.
- Отиди на **Elements**.
- Намери Age input-а.
- Копирай ми HTML-а на целия Age блок — от:

```
<div class="mb-3">
```

до:

```
</div>
```

Искам да видя **какъв HTML реално генерира Razor**, защото `asp-for` може да е генерирал нещо различно от това, което очакваме. Не пипаме CSS засега. Само ми дай генерирания Age блок и ще го оправим с една малка промяна.

- Вие казахте:**

```
<div class="mb-3"> <label class="form-label"
for="Age">Age</label> <input type="number" min="18" max="100"
class="form-control" data-val="true" data-val-range="The field Age
must be between 18 and 100." data-val-range-max="100" data-val-
range-min="18" data-val-required="The Age field is required."
id="Age" name="Age" value="0"> <input name="__Invariant"
type="hidden" value="Age"> <div class="age-hint"> Age → must be
between 18 and 100. </div> <span class="text-danger field-
validation-valid" data-valmsg-for="Age" data-valmsg-
replace="true"></span> </div>
```

4. ChatGPT каза:

Да — сега вече виждам проблема. 👍

HTML-ът е малко по-различен, отколкото предположихме:

```
<input ... id="Age" name="Age" value="0">
<input name="__Invariant" type="hidden" value="Age">
```

```
<div class="age-hint">
```

Между Age input-а и .age-hint има **още един** <input>:

```
<input name="__Invariant" ...>
```

Затова този CSS:

```
input[name="Age"]:focus + .age-hint
```

не може да намери .age-hint като непосредствен следващ елемент.

Стъпка 8 — поправяме само CSS

Замени:

```
input[name="Age"]:focus + .age-hint {
    display: block;
}
```

с:

```
input[name="Age"]:focus ~ .age-hint {
    display: block;
}
```

Разликата е:

+ → непосредствено следващ елемент

~ → всеки следващ sibling елемент

При нашия HTML:

Age input

↓

__Invariant input

↓

age-hint

~ ще намери .age-hint, дори да има друг елемент между тях.

Не променяй нищо друго.

После:

1. Save.
2. Refresh на Create страницата.
3. Кликни в Age.

Трябва да видиш:

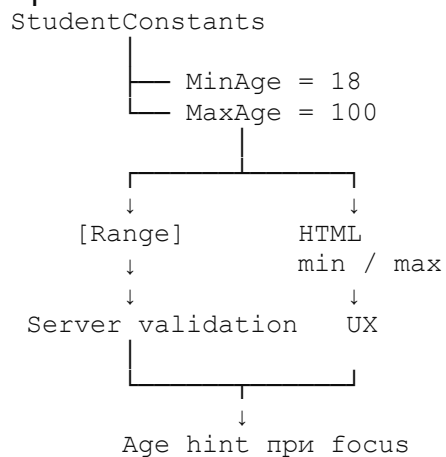
Age → must be between 18 and 100.

Кажете ми дали вече се появява.

1. стана и стандартните error съобщения ми харесват
2. **ChatGPT каза:**

Супер! 😊 Това според мен е още по-добър резултат.

Получихме точно това, което искахме, без да усложняваме проекта:



И стандартното:

The field Age must be between 18 and 100.

си е напълно достатъчно като error message. 👍

Така че **не ни трябва** ValidationMessages **за Age** и няма смисъл да създаваме custom validation.

Следващата стъпка

Сега можем да приложим **същия принцип към останалите полета**, но само там, където има реален validation rule.

Предлагам да продължим **едно поле наведнъж**, както досега.

Следва:

FirstName

Но преди да решим какво да добавим към него, трябва да видим какви ограничения има в Student модела.

Изпрати ми само:

Models/Student.cs

и ще продължим с **FirstName** — една малка промяна → **Build** → **тест**.

3. Вие казахте:

при грешка ми дават 2 различни: 1/Стойността трябва да е по-голяма или равна на 18 и 2/Стойността трябва да е по-малка или равна на 100

4. ChatGPT каза:

Да — и това всъщност е **интересно наблюдение**. 👍

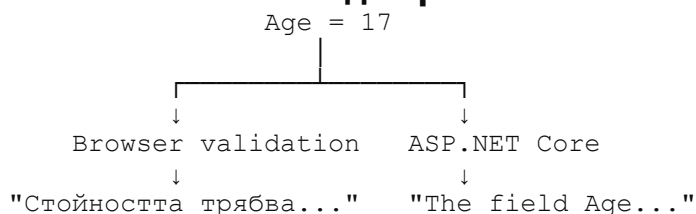
Това означава, че браузърът показва своите HTML validation съобщения за:

```
min="18"  
max="100"
```

а ASP.NET Core има отделно server-side validation чрез:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

Тоест в момента имаме **две различни системи за validation**:



И това е нормално.

Но тук има един UX въпрос

Ако при въвеждане на 17 браузърът спре submit-а и покаже: Стойността трябва да е по-голяма или равна на 18.

тогава потребителят **може изобщо да не стигне до ASP.NET Core validation message**.

А при 101 получава:

Стойността трябва да е по-малка или равна на 100.

Аз лично **харесвам тези съобщения**. Те са ясни и конкретни.

Така че не бих бързал да ги променяме.

Нашият информативен hint:

Age → must be between 18 and 100.

казва предварително какво се очаква.

А browser validation казва точно какво е нарушено:

Стойността трябва да е по-голяма или равна на 18.

Това е доста добро UX поведение.

Само едно нещо бих проверил

При Age = 17 виж дали съобщението се показва **до полето в нашия** `<span asp-validation-for="Age">`, или е browser popup/tooltip.

Ако е browser popup — всичко е наред.

Ако е в червения `<span>` — тогава ще знаем, че гледаме вече ASP.NET Core/jQuery validation и ще го разгледаме отделно. Но **не променяме нищо засега**. Кажи ми къде точно се появяват тези две съобщения.

1. под полето където беше и предишното което ние решихме да не използваме
2. **ChatGPT каза:**

Аха — **това е важно уточнение**. Тогава не говорим за browser роруп. Говорим за ASP.NET Core/jQuery client-side validation, която използва `[Range]` и показва съобщението в:

```
<span asp-validation-for="Age" class="text-danger"></span>
```

И тук вече виждам какво се е случило.

Когато имаме нашето:

```
ErrorMessage = ValidationMessages.AgeMustBeBetweenMinAndMax
```

получавахме **едно наше съобщение**.

След като го магнахме:

```
[Range(StudentConstants.MinAge, StudentConstants.MaxAge)]
```

`RangeAttribute` използва стандартното си съобщение. Но при теб то се показва като **две отделни validation съобщения** — едно за `minimum` и едно за `maximum`.

Това всъщност е напълно приемливо и аз бих го оставил така.

Например:

```
Age → must be between 18 and 100.
```

при `focus` — информативно.

А при грешка:

```
The value must be greater than or equal to 18.
```

```
The value must be less than or equal to 100.
```

Тоест:

Нормално състояние

↓

i Age → must be between 18 and 100.

Грешна стойност

↓

X The value must be...

Не бих връщал старото `ValidationMessages` **само заради това**.

Дже според мен сегашното решение е по-добро: потребителят предварително знае диапазона, а при грешка получава конкретно обяснение.

Следващата стъпка

Age вече можем да го считаме за **готово функционално**:

- [Range] ✓
- StudentConstants ✓
- HTML min/max ✓
- hint при focus без JS ✓
- стандартна validation грешка ✓
- без дублиране на 18/100 в кода ✓

Сега можем да преминем към **FirstName** и да направим същото според реалните му изисквания.

За него обаче първо трябва да видим Student.cs, за да не измисляме ограничения.

3. Вие казахте:

от browser е роруп

4. ChatGPT каза:

А, така вече е ясно. 👍

Тогава всичко е **точно както трябва**.

Това, което виждаш:

„Стойността трябва да е по-голяма или равна на 18.“

и

„Стойността трябва да е по-малка или равна на 100.“

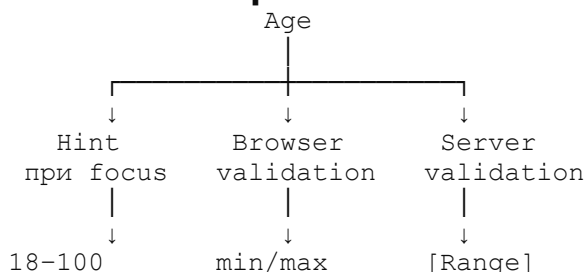
е **нативното HTML5 browser validation**, породено от:

```
min="18"
max="100"
```

То **не идва от ASP.NET Core** и не е в нашия:

```
<span asp-validation-for="Age">
```

Значи имаме три нива



И това е доста хубаво:

- ⓘ **Hint:** „Age → must be between 18 and 100.“
- 🌐 **Browser:** автоматично предупреждава при 17 или 101.
- 🛡️ **Server:** [Range] валидира независимо от браузъра.

Така че **не бих променял нищо по Age на този етап.**

И дори бих запазил browser съобщенията — те са ясни и идват без нито ред JavaScript.

Следваща малка стъпка

Сега можем да преминем към **FirstName**.

За да определим какви ограничения трябва да има там, изпрати ми Student.cs. Ще гледаме само FirstName и няма да променяме останалите полета още.